

Learn Claude Code

A progressive, open-source guide to building AI coding agents

ShareAI Lab | MIT License | github.com/shareAI-lab/learn-claude-code

Overview

Learn Claude Code is a comprehensive educational repository that teaches how to build AI coding agents from scratch, progressing through 12 sessions. Each session adds exactly one new mechanism on top of the core agent loop, mirroring how real tools like Claude Code and Cursor are constructed internally.

The central philosophy is: "The model is the agent. Our job is to give it tools and stay out of the way." The entire pattern is driven by one loop:

```
while stop_reason == 'tool_use':
    response = LLM(messages, tools)
    execute tools
    append results
    loop back
```

12 Progressive Sessions

Session	File	Focus	Motto
s01	s01_agent_loop.py	Core loop + bash tool	One loop & Bash is all you need
s02	s02_tool_use.py	Tool dispatch map	Adding a tool = adding one handler
s03	s03_todo_write.py	TodoManager / planning	An agent without a plan drifts
s04	s04_subagent.py	Fresh context for subtasks	Break big tasks; each gets clean context
s05	s05_skill_loading.py	On-demand knowledge inject	Load knowledge when needed, not upfront
s06	s06_context_compact.py	3-layer compression	Context fills up; you need to make room
s07	s07_task_system.py	File-based task graph	Break goals into tasks, persist to disk
s08	s08_background_tasks.py	Daemon threads	Run slow ops in background; keep thinking
s09	s09_agent_teams.py	Multi-agent teams	When too big for one, delegate to teammates
s10	s10_team_protocols.py	JSONL mailbox protocols	Teammates need shared communication rules
s11	s11_autonomous_agents.py	Self-claiming idle cycle	Teammates scan the board, claim tasks
s12	s12_worktree_task_isolation.py	Worktree isolation	Each works in its own dir, no interference

Learning Path

Phase 1 - The Loop (s01-s02)

Core agent loop and tool dispatch

Phase 2 - Planning & Knowledge (s03-s06)

Todo planning, subagents, skills, context compression

Phase 3 - Persistence (s07-s08)

Task graphs, background async execution

Phase 4 - Teams (s09-s12)

Multi-agent collaboration, protocols, worktree isolation

Directory Structure

- agents/ - 12 reference implementations (~4,600 lines of Python total)
- docs/en|zh|ja/ - Mental-model-first documentation in 3 languages
- skills/ - Reusable on-demand knowledge bases (agent-builder, code-review, mcp-builder, pdf)
- web/ - Next.js interactive platform with per-session visualizations
- requirements.txt - anthropic>=0.25.0, python-dotenv>=1.0.0

Technology Stack

Python backend

- anthropic - Claude API client
- python-dotenv - environment configuration

Web platform

- Next.js + React 19 + TypeScript
- Tailwind CSS + Framer Motion + Lucide React
- remark / rehype for Markdown & code highlighting

Key Design Principles

- One concept per session - each file adds exactly one mechanism
- Minimal code - only the essential pattern, no production boilerplate
- Mental-model-first docs - explains 'why' before 'how' with ASCII diagrams
- Progressive dependency - each session builds on the previous one
- Context isolation - subagents start fresh; worktrees prevent file conflicts

Quick Start

```
git clone https://github.com/shareAI-lab/learn-claude-code
cd learn-claude-code
uv sync
cp .env.example .env # add ANTHROPIC_API_KEY
uv run python agents/s01_agent_loop.py # start here
uv run python agents/s_full.py # capstone reference
cd web && npm install && npm run dev # interactive platform
```

The project is open-source under the MIT License and serves as the canonical reference for anyone wanting to understand or replicate the internals of modern AI coding agents.